Principles of Network Applications

IT5100D AY25/26 S2

NUS | Computing
National University of Singapore

© Copyright National University of Singapore. All Rights Reserved.

Network Applications

Programs that:

- Run on different end systems
- Communicate over network

Examples:

- Web browser
- Email clients
- Social media platforms
- File transfer applications
- Online games
- …

Two main architectures:

- Client-Server – most common
- Peer-to-Peer (P2P)

© Copyright National University of Singapore. All Rights Reserved.

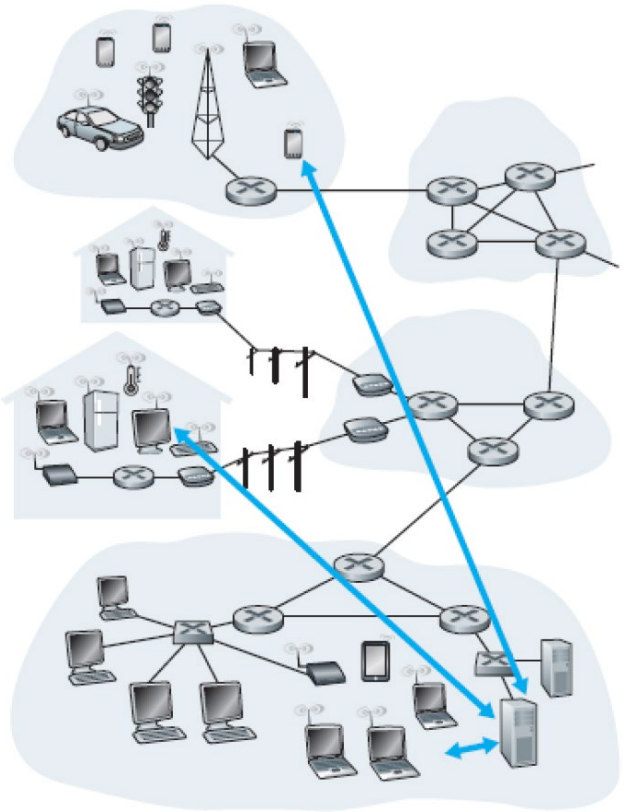
Client-Server Architecture

Server

- Always-on, wait for contact
- Fixed, known IP address
- Often reside on powerful machines (e.g., data centres)
- Can serve many clients simultaneously

Client

- Initiate contact, communicate with server
- May be intermittently connected
- May have dynamic IP addresses
- Do not talk directly to each other



Client-Server Architecture

- Formally, the terms client and server refer to the *programs* in those roles, not the computers running them
  - Although “server” is often used to refer to the computers themselves

- Possible for one computer to run multiple server programs for multiple services
- Possible for one computer to run server program(s) and client program(s)
- Possible for general-purpose computers to run server programs

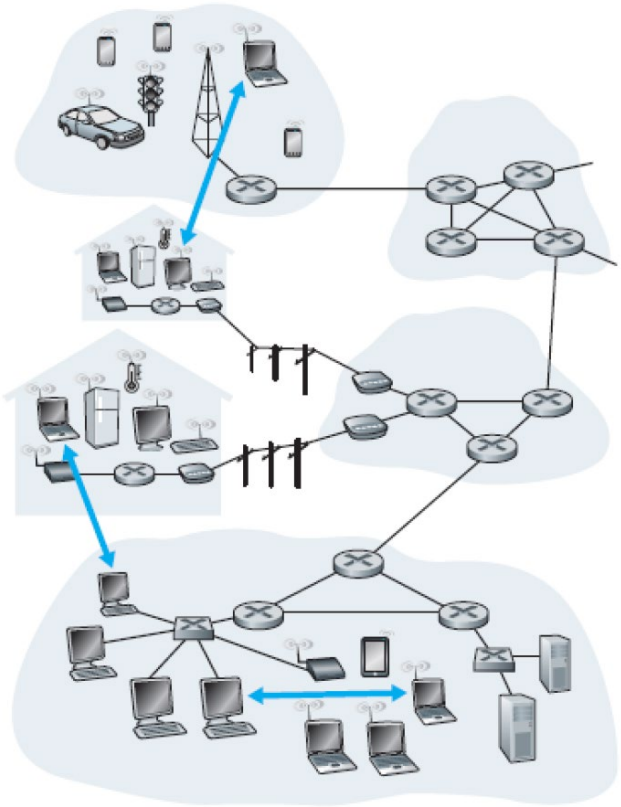
Peer-to-Peer (P2P) Architecture

No always-on server

Peers are arbitrary end systems, who:

- *Directly communicate with each other*
- Are intermittently connected and change IP addresses (complex management!)
- Request service from + provide service to other peers

Examples: P2P file sharing, Mesh instant messenger



Peer-to-Peer (P2P) Architecture

Pros:

- Self-scalability: new peers bring new service capacity along with new service demands
- Cost-effective: normally does not require significant server infrastructure and bandwidth

Cons:

- Security, performance, reliability challenges arising from decentralized structure

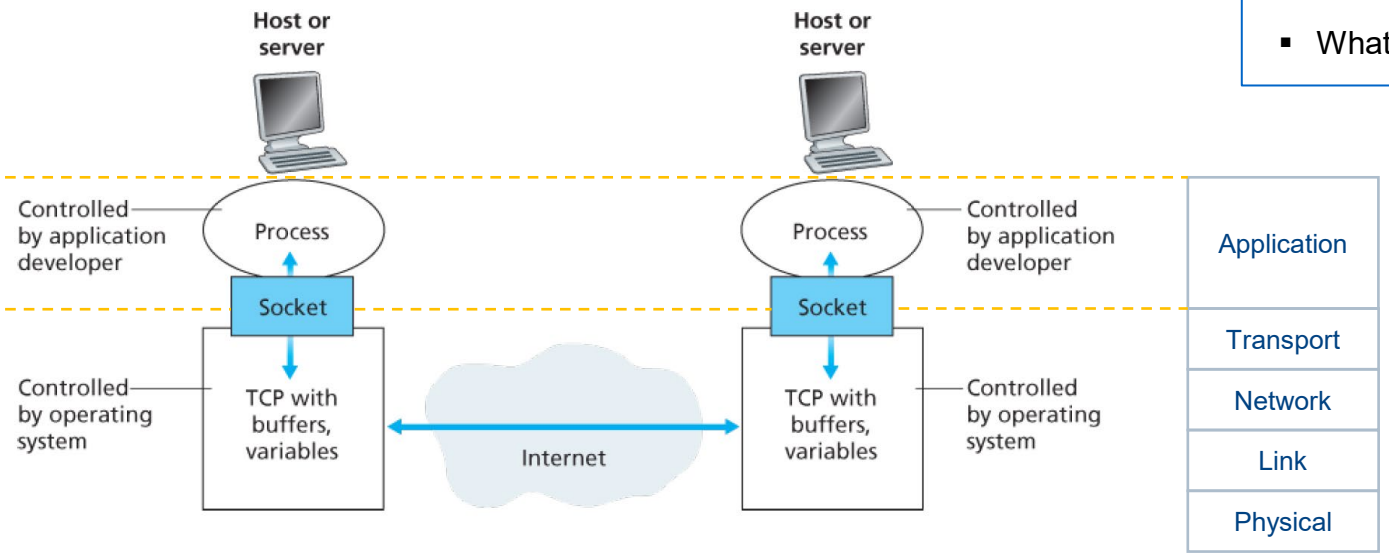
Hybrid architecture exists, e.g.:

- Some instant messaging apps use P2P for message exchange, with a centralised server to manage user accounts
- Some video conferencing apps (e.g., Zoom, Jitsi) provide P2P option for one-on-one meetings

Application-to-Network Interface

Process: program running within a host

- Processes within the *same host* communicate using **inter-process communication** (defined by the operating system)
- Processes in *different hosts* communicate by exchanging **messages**



Client-Server

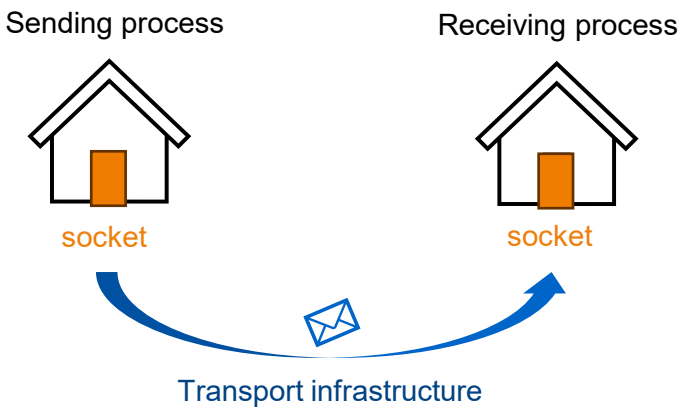
- Client process: process that initiates communication
- Server process: process that waits to be contacted

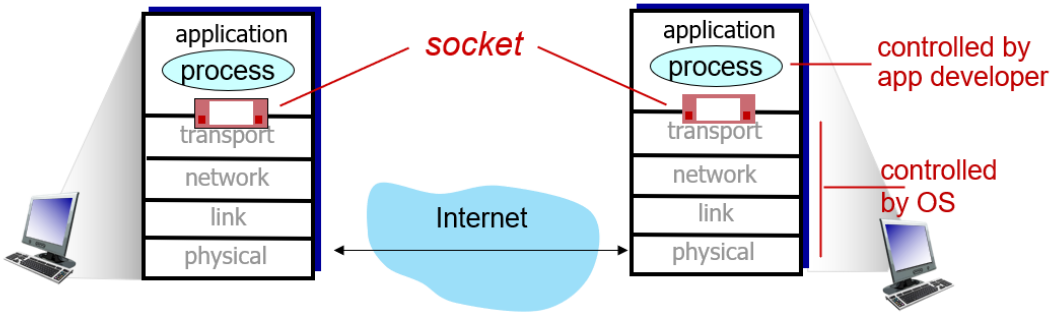
P2P

- What are the roles?

Sockets

Process sends/receives messages to/from its **socket.**



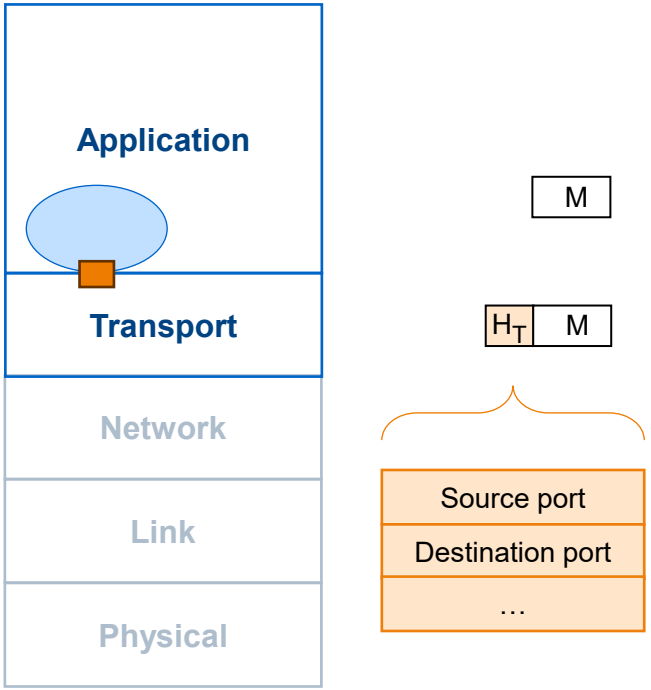


Addressing Processes

- To receive messages, process must have an identifier.
- Host device has a unique address (32-bit IP address) – is this sufficient?

- Many processes can be running on the same host.
- The identifier includes both **IP address** and the **port number** for the process on the host.

- IP addresses and port numbers are handled at the transport layer.

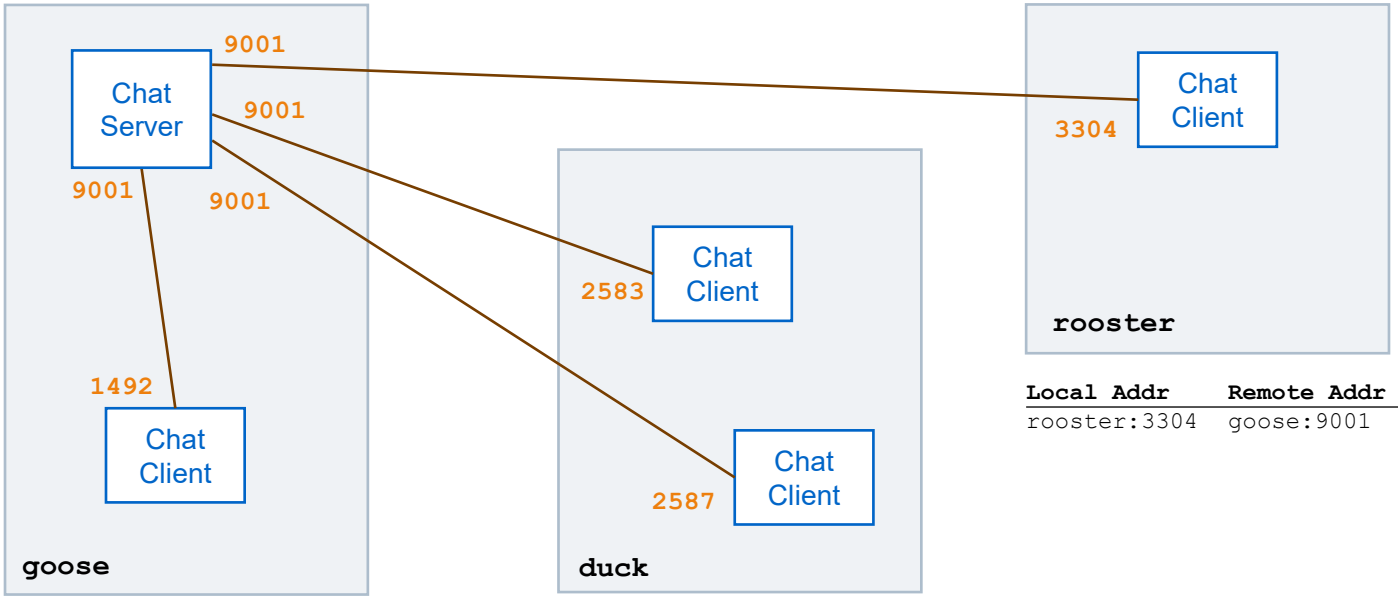


Port Numbers

- 16-bit values
- Port number assignment authority: Internet Assigned Numbers Authority (IANA)
- List of some common ports (courtesy of Jeremy Stretch)

Range	Designation	Explanation
`0...1023`	System or well-known ports	Used by system processes that provide widely used types of network services. Examples: `80` (HTTP), `443` (HTTPS), `22` (SSH)
`1024...49151`	User or registered ports	Assigned by IANA for specific service upon application by a requesting entity. Examples: `3306` (MySQL), `3724` (some Blizzard games)
`49152...65535`	Dynamic, private or ephemeral ports	Unrestricted – cannot be registered with IANA. Used for private or customized services, temporary purposes, or automatic allocation of ephemeral ports.

Example



Local Addr	Remote Addr	State
rooster:3304	goose:9001	ESTABLISHED

Local Addr	Remote Addr	State
goose:9001	goose:0	LISTENING
goose:9001	duck:2583	ESTABLISHED
goose:9001	duck:2587	ESTABLISHED
goose:9001	rooster:3304	ESTABLISHED
goose:9001	goose:1492	ESTABLISHED
goose:1492	goose:9001	ESTABLISHED

Local Addr	Remote Addr	State
duck:2583	goose:9001	ESTABLISHED
duck:2587	goose:9001	ESTABLISHED

How Do Servers Handle Multiple Clients?

- Servers do not know in advance how many or who the clients are
- To handle multiple simultaneous clients, a server should either:

Be multi-threaded

```
while (true) {
    Connection c = waitForConnection();
    spawnANewThreadToHandle(c); // or submit to task to thread pool
}
```

Be event-driven

```
server.on('connect', (connection) => {
  connection.on('somemessage1', (data) => handlerForMessage1());
  ...
  connection.on('somemessageN', (data) => handlerForMessageN());
}
```

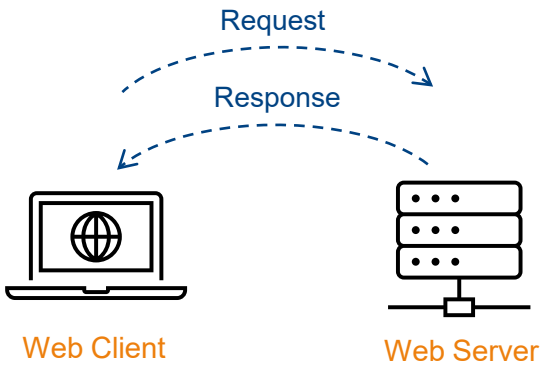
Web Application Structure

IT5100D AY25/26 S2

NUS | Computing
National University of Singapore

© Copyright National University of Singapore. All Rights Reserved.

Web Terminology



where the web objects are

collection of web pages

By convention…

Web site:

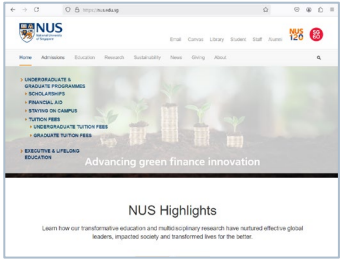
Server always sends back the same data *without any user interaction*.

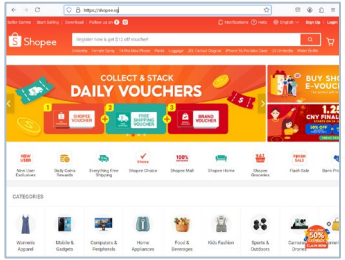
Web app:

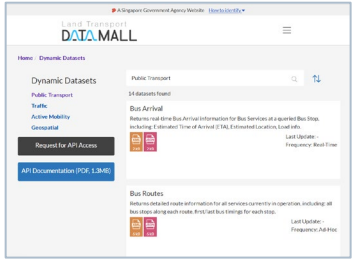
User interaction (click/tap, keypress, ...) affects program execution.

Web service:

Server only produces raw data (text, JSON, …) e.g., utilized by another web server or mobile app.







Web Application and Internet Protocols

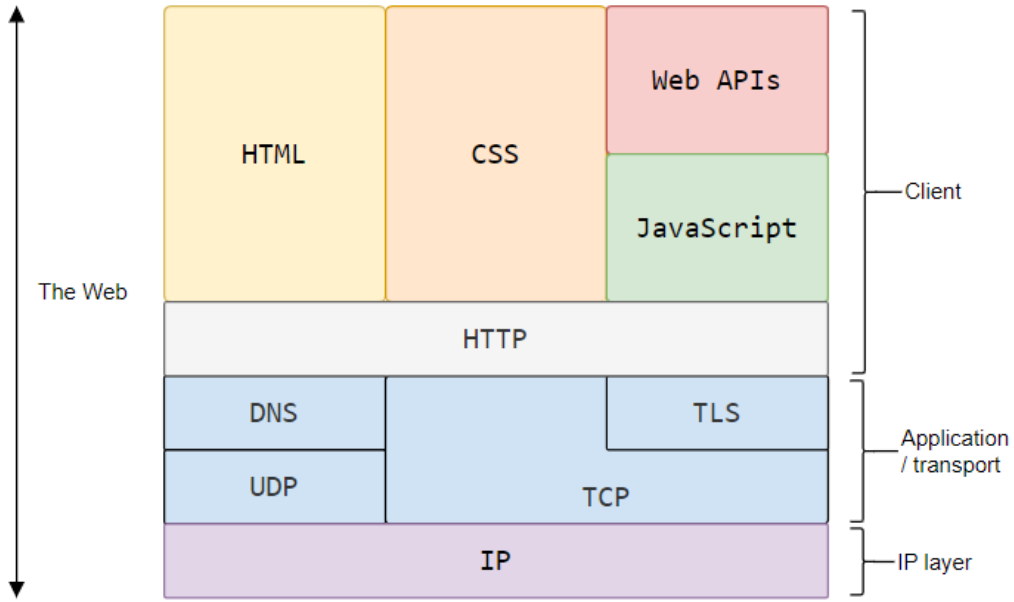


Image source: MDN Web Docs

Simple web apps

- HTML to describe *content*
- CSS to describe *presentation*
- JavaScript to program the *interactivity*
- Resources: Images, video, text, other info

All these are web objects

- Possible to store on different web servers
- Fetched and rendered by the web browser (a client)

Scaling up…

- External APIs
- Storage to keep track of stuff

Uniform Resource Identifier (URI)

- A web page is typically a base HTML file + several referenced resources
- URI is a formatted string used to describe a resource
  - Uniform: Same overall format no matter the type or how it is located / accessed

- The URI syntax is federated and extensible
  - Generic syntax described in RFC 3986
  - Begins with a scheme name that determines the specification
- Individual RFCs describe particular schemes
  - E.g., URI section in RFC 2616 (HTTP)

```
ftp://ftp.is.co.za/rfc/rfc1808.txt

http://www.ietf.org/rfc/rfc2396.txt

ldap://[2001:db8::7]/c=GB?objectClass?one

mailto:John.Doe@example.com

news:comp.infosystems.www.servers.unix

tel:+1-816-555-1212

telnet://192.0.2.16:80/

urn:oasis:names:specification:docbook:dtd:xml:4.1.2
```

URI, URL, and URN

URIs can be used to name things, to locate things, or both.

You may encounter:

- URL (uniform resource locator): URIs that describe how to locate the resource.
- URN (uniform resource name): URIs with the **urn** scheme.

Just because something has a URI, it does not mean you can find it or even access it!

Running a Web Server on Your Computer

Convenient ways:

- Node.js http-server module
- Python http.server module
- …

Step by step guide from MDN Web Docs

Making the web server accessible publicly requires extra steps – more on this when we get to *IP Addressing* part

Useful tools

- netstat: view active connections on your system
- Browser DevTools: view objects transmitted etc.
- Wireshark: *Adapter for loopback traffic capture* interface for “self” connections